Two Trees

A design for delegated voting: the algebra, the types,
and the one thing the formalism refuses to decide

Claude, 14 September 2026

Design only. Nothing here is built; the point of writing it first is that one of the claims in `README.md` turned out to be wrong as soon as the second tree was taken seriously, and it is cheaper to find that here.

The brief is in `PROBLEM.md`. In one line: every citizen votes on every issue, nobody has time, so a vote is a *delegation* — refinable issue by issue, all the way down — and the finest issues are settled in chat rooms where a delegate's vote counts once per citizen who sent it to them.

1 One container

$$\text{IssueC} \;=\; \text{Issue} \triangleleft \text{Vote}$$

A shape is an issue; a position is a valid vote *on that issue*. The dependency is not decorative. A referendum admits aye or nay, a seat admits a candidate from a list, a budget line admits a number in a range, and a vote of the wrong kind is not a dissenting vote but a type error.

A citizen is a `Runner<IssueC>`: somebody an issue can be put to, who answers. That is the entire ontology.

"Regular citizen", "politically active citizen" and "political party" are not three types. They are three ways of being a runner — answering from your own head, answering by asking one other, answering by asking several and combining. The type cannot tell them apart, and should not, because that is what makes becoming politically active a decision rather than a migration.

2 The two trees

Both trees carry the same forward map and differ entirely in the backward one. That is the design.

The delegation tree

Rooted at a citizen. Its skeleton is the *issue* tree — government, then health, education and military, then their sub-issues — and a delegate is attached at whichever nodes the citizen has bothered to assign. Everything below an assignment inherits it until overridden.

```
(citizen)
|- health ................... party C
|- education ................ party A
|    |- early childhood ..... (inherits A)
|    |- STEM ................ (inherits A)
|    |- art and creativity .. Sarah Smith      <- overrides, this subtree only
```

```
|    '- physical education .. (inherits A)
'- military ................. party C
```

So a delegation is a **partial function on the issue tree, extended downward by inheritance**. The total function it denotes is on leaves; the partial function is its presentation; and the lazy delegation — party B for everything — is the constant function, presented by one assignment at the root.

Exactly one delegate answers any given issue. The backward map is therefore the **identity**: what comes back is my vote, unaltered. Delegation is a *vertical* morphism, which `federation` calls a degenerate corner of the formalism. Here the degenerate corner is the subject.

The party tree

Rooted at a party. Its children are its members, and it too is indexed by issue, because a party may put defence to one member and health to another.

```
(party A)
|- education
|    |- art and creativity .. member: Ali
|    '- (everything else) ... members: Ali, Bea, Carl   -> amalgamate
'- health .................. members: Bea, Carl        -> amalgamate
```

Here several members may answer the same issue, and the party must return *one* vote. The backward map is therefore an **amalgamation**: a fold over the replies — majority, weighted majority, unanimity, or a party line that ignores them.

	delegation tree	party tree
rooted at	a citizen	a party
indexed by	the issue tree	the issue tree
forward map	route by issue	route by issue
how many answer	exactly one	possibly several
backward map	**identity**	**a fold**
combinator	$+$	$\boxtimes$

Observation. *The two trees are one construction with two backward maps. This is the same finding* `federation.pdf` *reports for cyclic federations — "the obstruction is in the response direction … the failure is entirely in the backward map" — arrived at from a different subject. Forward maps are where the routing lives and they are easy. Backward maps are where the content is.*

3 The algebra, and all four combinators have a job

	in the algebra	here
$+$	disjoint shapes, each keeping its own positions	**delegation.** One delegate per issue; refinement is which summand
$\boxtimes$	both shapes up front, **both replies owed**	**amalgamation.** Every member polled, every reply owed, then folded
$\times$	both offered, **exactly one answered**	**abstention.** A second choice, used when the first does not answer
$\triangleleft$	the first reply chooses the second prompt	**chaining.** You to a party, the party to a member

This is the first of these projects in which all four are load-bearing. `stalin` and `federation` never use +; `firm` uses it once. Here + is the primary structure and ⊠ is what makes a party a party.

4 The types

```
// ── Issues ──────────────────────────────────────────────────────────
type IssueId = string;              // a path: "education/art"
type Kind =
  | { tag: "Binary" }
  | { tag: "Choice"; options: readonly string[] }
  | { tag: "Amount"; lo: number; hi: number };

interface Issue { id: IssueId; kind: Kind; children: readonly IssueId[] }

// ── The container. The position depends on the issue's kind. ────────
type VoteFor<K extends Kind> =
    K extends { tag: "Binary" } ? { tag: "Aye" } | { tag: "Nay" }
  : K extends { tag: "Choice" } ? { tag: "Pick"; option: string }
  : K extends { tag: "Amount" } ? { tag: "Amount"; value: number }
  : never;

type IssueC =
  | Fib<{ tag: "Poll"; issue: IssueId; visited: readonly VoterId[] }, Ballot>
  | Fib<{ tag: "Reset" }, { tag: "ResetAck" }>;

// A ballot ADMITS FAILURE. Cycles are a reply, not an exception, so the
// caller must handle them and the type says so.
type Ballot =
  | { tag: "Cast";    vote: Vote;   weight: number }
  | { tag: "Cycle";   ring: readonly VoterId[] }
  | { tag: "Abstain"; who: VoterId; why: string };

// ── Delegation: a partial map, inherited downward ───────────────────
type Delegation = ReadonlyMap<IssueId, VoterId>;

/** The nearest ancestor of `issue`, inclusive, that has an entry. */
function delegateFor(d: Delegation, issue: IssueId): VoterId | null;

// ── Parties ─────────────────────────────────────────────────────────
type Rule =
  | { tag: "Majority" }                // one member, one vote
  | { tag: "Weighted" }                // by delegated weight
  | { tag: "Unanimous" }               // or the party abstains
  | { tag: "Line"; vote: Vote };       // a leaf wearing a party's coat

interface Party { id: VoterId; members: readonly VoterId[]; rule: Rule;
                  assign: Delegation }   // which members get which issues
```

Two remarks on the types rather than the design.

`Ballot` admitting `Cycle` is deliberate. A cycle is an ordinary outcome of asking a question, not an error condition, and putting it in the position type forces every caller to say what it does about one.

`Rule` having a `Line` constructor is what lets a party be a leaf without being a different type. A party with a line answers from its own head; the citizen delegating to it sees the same interface either way, and *ought* to be able to see which it is — which is an argument for exposing `rule`, not for a second type.

5 Resolution, and a correction

To resolve a vote is to walk from the citizen, following the forward maps, until somebody answers. The state — who has been visited — rides in the prompt, because `federation/mobius.ts` establishes that a header cannot be a type and `visited` must be typed if the base case is to be checkable.

A claim in `README.md` is wrong, and the party tree is what breaks it.

It said: per issue each citizen delegates to at most one representative, so out-degree is at most one, the graph is *functional*, every component is a tail running into a single cycle, and cycle detection is correspondingly cheap.

That holds for the delegation tree alone. It fails the moment a party is involved, because a party polls several members. Out-degree is one at a citizen and n at a party, so the combined structure is not functional and its components are not ρ-shaped.

Resolution is therefore a fold over a tree, not a walk down a path, cycle detection must be per branch rather than per component, and the same delegate may be reached twice by different routes — which is a question about weights before it is a question about cycles.

That correction is the reason this document exists before the code does.

6 Weight

A delegate's weight on an issue is the number of citizens whose resolution reaches them on that issue. So weight is an **output of resolution, not an input to it**, and it is per-issue: the same delegate carries different weight on health and on defence.

The re-convergence just noted has a consequence here. If a citizen reaches Ali directly *and* through party A, Ali must not be counted twice for that citizen. Weight is therefore a count of *citizens reached*, deduplicated at the source, rather than a sum accumulated along edges.

7 What the algebra refuses to decide

Two choices are political rather than mathematical, and the formalism should present them rather than make them.

Cycle policy. Detection is settled. What should happen is not. The vote abstains; or falls back to a $\times$ second choice; or returns to the citizen as a notification and waits; or is divided evenly round the ring. These are four different theories of what a delegated vote *is*, and the type `Ballot` is deliberately wide enough to express all four.

Amalgamation rule. Whether a party's members are counted one each or by the weight they personally carry is the difference between a party as a club and a party as a federation of blocs. `Rule` makes it a per-party choice, and it should probably be visible to anyone delegating to that party.

8 What to build first

1. The issue tree, and `delegateFor` with inheritance. Small, and everything rests on it.

2. `resolve(citizen, issue)` as a tree fold with `visited` in the prompt, returning a `Ballot`. Cycle policy pluggable, defaulting to abstain, so the argument can be had against running code.
3. Weights, deduplicated at the source, and a tally for one leaf issue.
4. Only then: parties as separate processes, one per delegate, and the chat room as $\boxtimes$ over the participants.

9 What is assumed

That a worker's state may be a set, since `visited` rides in the prompt exactly as `fuel` does in `mobius`. That question is out to Strathclyde and unanswered.

That the electorate may be unbounded. The argument is in `README.md`: every delegate serves the *same* container, so $\Sigma\,(d : D).\,\mathrm{IssueC} \cong D \times \mathrm{IssueC}$, and a sum of a constant family is a product with its index set and does not need the index finite. This is the one place where voting is easier than the firm, and it is worth knowing what would destroy it: give parties a capability individuals lack — a manifesto endpoint, a whip — and the family stops being constant.